Time and Space Complexity

Agenda

- Introduction to Time Complexity
- Common Time Complexities
- Analyzing Time Complexities
- Importance of Time Complexity Analysis
- Time Complexity of Operations
- Introduction to Space Complexity
- Common Space Complexities
- Analyzing Space Complexities
- Importance of Space Complexity Analysis
- Space Complexity of Data Structures
- Space Complexity in Algorithms

Introduction to Time Complexity:

Time complexity is a measure of the amount of time an algorithm takes to complete its execution as a function of the size of its input. It helps in assessing how the runtime of an algorithm grows with increasing input size.

Key Points:

1. Asymptotic Analysis: Time complexity is often expressed using asymptotic notation (Big O, Omega, Theta) to describe the upper, lower, and tight bounds on the growth rate of an algorithm's runtime.
2. Worst-case scenario: Describes the maximum number of operations an algorithm performs for any input size.
3. Average-Case Complexity: Describes the expected number of operations averaged over all possible inputs.
4. Best-Case Complexity: Describes the minimum number of operations an algorithm performs for any input size.
5. Comparison Basis: Time complexity enables the comparison of algorithms to determine which one is more efficient for a given problem.

Common Time Complexities:

O(1) Constant Time Complexity:

Description: The algorithm's runtime remains constant, irrespective of the input size.
Example: Accessing an element in an array by index, performing basic arithmetic operations.

Python:

```
def main():
    x = 5
    y = 10
    result = x + y  # Constant time operation
    print("Result:", result)

if __name__ == "__main__":
    main()
```

O(log n) Logarithmic Time Complexity:

Description: The algorithm's runtime grows logarithmically with the input size.
Example: Binary search, certain tree operations like finding the height of a balanced binary search tree.

Python

```
def main():
    arr = [1, 3, 5, 7, 9, 11]
    target = 7
    low, high = 0, len(arr) - 1
    while low <= high:
        mid = low + (high - low) // 2
        if arr[mid] == target:
            print("Index of", target, ":", mid)  #
Logarithmic time operation
```

```
            break
        elif arr[mid] < target:
            low = mid + 1
        else:
            high = mid - 1

if __name__ == "__main__":
    main()
```

O(n) Linear Time Complexity:

Description: The algorithm's runtime increases linearly with the input size.

Example: Linear search, traversing through an array or list, counting the occurrences of an element in a list.

Python:

```
def main():
    arr = [1, 3, 5, 7, 9, 11]
    target = 7
    for i in range(len(arr)):
        if arr[i] == target:
            print("Index of", target, ":", i)  # Linear time
operation
            break

if __name__ == "__main__":
    main()
```

O(n log n) Linearithmic Time Complexity:

Description: The algorithm's runtime grows in proportion to n multiplied by the logarithm of n.

Example: Merge sort, heap sort, certain tree operations like sorting a binary search tree.

Python

```
def main():
    arr = [4, 2, 1, 3, 5]
    arr.sort()  # Linearithmic time operation (using
list.sort)

if __name__ == "__main__":
    main()
```

O(n^2) Quadratic Time Complexity:

Description: The algorithm's runtime grows quadratically with the input size.
Example: Bubble sort, selection sort, certain dynamic programming algorithms like the naive implementation of the knapsack problem.

Python

```
def fib(n):
    if n <= 1:
        return n
    return fib(n - 1) + fib(n - 2)  # Exponential time
operation

def main():
    n = 10
    result = fib(n)

if __name__ == "__main__":
    main()
```

O(n^k) Polynomial Time Complexity:

Description: The algorithm's runtime grows polynomially with the input size, where k is a constant.
Example: Matrix multiplication using the naive approach, certain graph algorithms like the FloydWarshall algorithm.

Python

```
def main():
    A = [[1, 2], [3, 4]]
    B = [[5, 6], [7, 8]]
    result = [[0, 0], [0, 0]]

    # Polynomial time operation (Matrix Multiplication)
    for i in range(2):
        for j in range(2):
            for k in range(2):
                result[i][j] += A[i][k] * B[k][j]

if __name__ == "__main__":
    main()
```

O(2^n) Exponential Time Complexity:

Description: The algorithm's runtime doubles with each increase in the input size.
Example: Generating all subsets of a set, certain recursive algorithms with exponential growth.

Python

```
def fib(n):
    if n <= 1:
        return n
    return fib(n - 1) + fib(n - 2)  # Exponential time
operation

def main():
    n = 10
    result = fib(n)
```

```
if __name__ == "__main__":
    main()
```

O(n!) Factorial Time Complexity:

Description: The algorithm's runtime grows factorial with the input size.

Example: Solving the traveling salesman problem using brute force, certain permutation and combination problems.

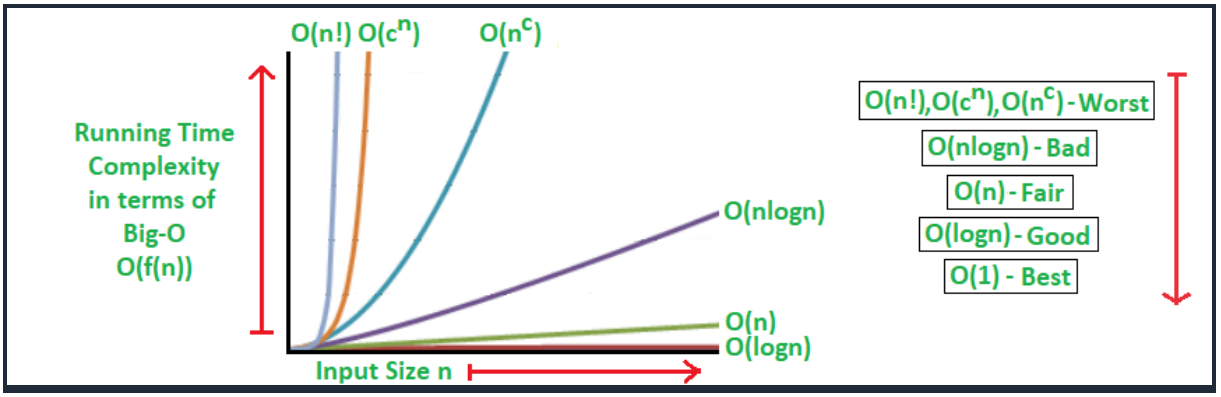
Python

```
def main():
    arr = [1, 2, 3]
    n = len(arr)

    # Factorial time operation (Permutations)
    def next_permutation(arr):
        i = len(arr) - 2
        while i >= 0 and arr[i] >= arr[i + 1]: i -= 1
        if i < 0: return False
        j = len(arr) - 1
        while arr[j] <= arr[i]: j -= 1
        arr[i], arr[j] = arr[j], arr[i]
        arr[i + 1:] = reversed(arr[i + 1:])
        return True

    while next_permutation(arr): print(*arr)

if __name__ == "__main__": main()
```



Analyzing Time Complexities:

Basic Operations:
Analyze the number of basic operations executed by the algorithm as a function of the input size.

Looping Constructs:
Identify nested loops and analyze the number of iterations performed by each loop.

Recursion:
For recursive algorithms, formulate a recurrence relation to express the algorithm's time complexity.

Importance of Time Complexity Analysis

Efficiency Comparison:
Compare the efficiency of different algorithms to select the most suitable one for a given problem.

Scalability Assessment:
Assess how well an algorithm scales with increasing input size to determine its suitability for large datasets.

Optimization Opportunities:
Identify parts of the algorithm with high time complexity for optimization to improve overall performance.

Time Complexity of Operations

Array Operations:

- Accessing an element by index: O(1)
- Searching (linear search): O(n)
- Sorting (quicksort, mergesort): O(n log n)

Linked List Operations:

- Accessing an element: O(n)
- Insertion/deletion at the beginning/end: O(1)
- Insertion/deletion at an arbitrary position: O(n)

Tree Operations:

- Searching/insertion/deletion (in balanced binary search tree): O(log n)
- Traversal (inorder, preorder, postorder): O(n)

Hash Table Operations:

- Searching/insertion/deletion (average case): O(1)
- Worst-case scenario (with collisions): O(n)

Sorting Algorithms:

Bubble Sort:

- Worst Case: O(n^2)
- Average Case: O(n^2)
- Best Case: O(n) (when the array is already sorted)

Selection Sort:

- Worst Case: O(n^2)
- Average Case: O(n^2)
- Best Case: O(n^2)

Insertion Sort:

- Worst Case: O(n^2)
- Average Case: O(n^2)
- Best Case: O(n) (when the array is already sorted)

Merge Sort:

- Worst Case: O(n log n)
- Average Case: O(n log n)
- Best Case: O(n log n)

Quick Sort:

- Worst Case: O(n^2)
- Average Case: O(n log n)
- Best Case: O(n log n)

Heap Sort:

- Worst Case: O(n log n)
- Average Case: O(n log n)
- Best Case: O(n log n)

Counting Sort:

- Worst Case: O(n + k)
  n is the number of elements in the input array.
  k is the range of input.

Radix Sort:

- Worst Case: O(n k)
  n is the number of elements in the input array.
  k is the maximum number of digits in the largest number.

Sorting Algorithm	Best Case	Average Case	Worst Case
Insertion	O(n)	$O(n^2)$	$O(n^2)$
Selection	$O(n^2)$	$O(n^2)$	$O(n^2)$
Bubble	$O(n^2)$	$O(n^2)$	$O(n^2)$
Heap	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$
Merge	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$
Quick	$O(n \log n)$	$O(n \log n)$	$O(n^2)$

Searching Algorithms:

Linear Search:

- Worst Case: O(n)
- Average Case: O(n)
- Best Case: O(1) (when the element is found at the beginning)

Binary Search:

- Worst Case: O(log n)
- Average Case: O(log n)
- Best Case: O(1) (when the element is found at the middle)

Interpolation Search:

- Worst Case: O(n) if elements are uniformly distributed
- Average Case: O(log log n) if elements are uniformly distributed
- Best Case: O(1) if the element is located at the beginning

Jump Search:

- Worst Case: O(sqrt(n))
- Average Case: O(sqrt(n))
- Best Case: O(1) (when the element is found at the beginning)

Exponential Search:

- Worst Case: O(log n)
- Average Case: O(log n)
- Best Case: O(1) (when the element is found at the beginning)

Introduction to Space Complexity:

Space complexity is a measure of the total amount of memory an algorithm needs to run to completion as a function of the size of the input. It includes memory used by variables, data structures, function calls, and auxiliary space.

Key Concepts:

1. **Auxiliary Space**: The extra space or temporary space used by the algorithm excluding input space.

2. **Input Space**: Memory required to store the input data.

3. **Total Space** = Auxiliary Space + Input Space.

4. **Scalability**: Space complexity helps determine how well an algorithm scales with increasing input sizes.

5. **Trade-Offs**: Often there's a trade-off between time and space complexity.

Common Space Complexities:

O(1) Constant Space Complexity

Description: Space remains constant regardless of input size.

Python

```
def add(a, b):
    result = a + b
    return result  # Constant space
```

O(log n) Logarithmic Space Complexity:

Description: Usually seen in recursive algorithms like binary search.

Python

```
def binary_search(arr, low, high, target):
    if low > high:
        return -1
    mid = (low + high) // 2
    if arr[mid] == target:
        return mid
    elif arr[mid] > target:
        return binary_search(arr, low, mid - 1, target)
    else:
        return binary_search(arr, mid + 1, high, target)
```

O(n) Linear Space Complexity:

Description: Space grows proportionally to input size.

Python

```
def copy_array(arr):
    return arr.copy()  # Linear space
```

O(n^2) Quadratic Space Complexity:

Description: Typically appears with 2D matrices.

Python

```
def create_matrix(n):
    return [[0 for _ in range(n)] for _ in range(n)]
```

O(2^n) Exponential Space Complexity:

Description: Seen in recursive subset or permutation generation.

Python

```
def subsets(nums):
    if not nums:
        return [[]]
    tail = subsets(nums[1:])
    return tail + [[nums[0]] + x for x in tail]
```

Analyzing Space Complexity:

1. **Primitive Variables**: Count the number of basic variable declarations.
2. **Dynamic Structures**: Track the size of arrays, lists, dictionaries.
3. **Recursion**: Stack space used in recursive function calls.
4. **Loop-Based Growth**: Memory allocated within loops grows with iterations.

Importance of Space Complexity Analysis:

- **Memory-Constrained Environments**: Crucial in embedded systems or mobile devices.
- **Algorithm Efficiency**: Helps in identifying memory bottlenecks.
- **Time-Space Trade-Offs**: Use space to save time and vice versa.
- **Performance Optimization**: Reducing memory use can improve cache performance.

Space Complexity of Data Structures:

Structure	Average Space Complexity
Array (size n)	O(n)

Linked List (n nodes)	O(n)
Stack/Queue	O(n)
Hash Table (n items)	O(n)
Binary Tree (n nodes)	O(n)
Graph (adjacency list)	O(V + E)
Graph (adjacency matrix)	$O(V^2)$

Space Complexity in Algorithms:

Sorting Algorithms

Algorithm	**Space Complexity**
Bubble Sort	O(1)
Insertion Sort	O(1)

Merge Sort	O(n)
Quick Sort	O(log n) (stack)
Heap Sort	O(1)

Searching Algorithms

Algorithm	**Space Complexity**
Linear Search	O(1)
Binary Search	O(log n)
DFS (recursive)	O(h)
BFS (queue)	O(n)